

Daily Scholar Papers Report — 2026-04-26

2026-04-26

Daily Scholar Papers Report — 2026-04-26

[Download PDF](#)

Window covered: 2026-04-13 → 2026-04-25 (Google Scholar alerts) **Pipeline:** Source-priority triage → Stage-1 Skip/Proceed → Stage-2 deep-read with meta-cognition.

0. Executive Summary

This window’s haul is dominated by three currents. (1) LLM-agentic software analysis is consolidating from a “can it work?” question into a benchmarking question — Pradel’s group ships *AnalysisBench*, and David Lo’s group reports a 100+ CVE field outcome from agentic vulnerability discovery. (2) Path/program-analysis classics are being made scalable for modern workloads — Qingkai Shi’s *Hermes* tackles path-sensitive pointer analysis under sparse value-flow, and Hakjoo Oh’s *Prunario* tackles redundancy in AV scenario testing. (3) A clean piece of systems security comes from Zhiqiang Lin’s group: a token-replay break of Apple Intelligence with CVE assigned.

Outstanding (Stage-2 deep-read): 4 Keep (Stage-2 deep-read): 5 Borderline High-Priority: 2

The full ranking and analyses for the highlighted set follow.

1. Highlighted Papers (Ranked)

#	Title	Followed Author / Source	Venue / Source	Decision
1	Hermes: Making Path-Sensitive Pointer Analysis Scalable for Sparse Value-Flow Analysis	Qingkai Shi	PACMPL (PLDI/OOPSLA-class) 2026	Outstanding
2	Prunario: Testing Autonomous Driving Systems by Pruning Likely Redundant Scenarios	Hakjoo Oh	PACMPL 2026	Outstanding
3	Too Private to Tell: Practical	Zhiqiang Lin	arXiv (sec, CVE issued) 2026	Outstanding

#	Title	Followed Author / Source	Venue / Source	Decision
	Token Theft Attacks on Apple Intelligence			
4	Evaluating LLM Agents on Automated Software Analysis Tasks (AnalysisBench / AnalysisAgent)	Michael Pradel	arXiv 2026	Outstanding
5	TitanCA: Lessons from Orchestrating LLM Agents to Discover 100+ CVEs	David Lo	arXiv 2026	Keep
6	OS-SANITIZER: System-wide Latent Defect Inference in Linux Applications	Thorsten Holz	preprint 2026	Keep
7	Determining the Unreachable: Constraint-Guided Reachability Analysis for Dependency Vulnerabilities	Recommended (PACMPL)	PACMPL 2026	Keep (Borderline High-Priority)
8	V2E: Validating Smart Contract Vulnerabilities through Profit-driven Exploit Generation and Execution	Zibin Zheng	arXiv 2026	Keep
9	Fragile Deliveries: Inconsistencies in Android Parcel and Their Security Consequences	Zhiqiang Lin	preprint 2026	Keep
10	Denoising Fault Localization with	Andreas Zeller	preprint 2026	Keep

#	Title	Followed Author / Source	Venue / Source	Decision
	Test Line Proximity			
11	Capturing Monetarily Exploitable Vulnerability in Smart Contracts (FAUDITOR)	Recommended (Kangjie Lu)	arXiv 2026	Borderline High-Priority
12	Cracking Federated Privacy: Initialization-Resilient Gradient Inversion	Zibin Zheng (co-author chain)	preprint 2026	Borderline High-Priority

2. Outstanding Papers (Deep-Read)

2.1 Hermes: Making Path-Sensitive Pointer Analysis Scalable for Sparse Value-Flow Analysis

Paper

- **Title:** Hermes: Making Path-Sensitive Pointer Analysis Scalable for Sparse Value-Flow Analysis
- **Authors:** Y. He, R. Jiang, H. Zhang, Q. Shi, H. Huang, R. Wu
- **Venue / Source:** Proceedings of the ACM on Programming Languages (PACMPL) 2026 — i.e., PLDI/OOPSLA-class
- **Year:** 2026

Priority Assessment

- **Source Priority:** High — Qingkai Shi is on the followed list and a leading voice in scalable static analysis (Pinpoint, FALCON). PACMPL is top-tier.
- **Reason:** Both author and venue are in the high-priority bucket; topic (sparse value-flow with path sensitivity) directly intersects the user's program-analysis interest.

Screening Decision

- **Stage 1 Decision:** Proceed
- **Reason:** Title states a precise scalability claim about a precision-cost combination (path-sensitive + sparse value-flow) that has historically been intractable. High research value, high methodology reusability for any vuln-detection / data-flow tool.
- **Research Value:** High

- **Methodology Reusability:** High

Quality Decision

- **Decision:** Outstanding
- **Reason:** PACMPL acceptance + a long-standing scalability target + Shi's track record. Even at the abstract level the contribution is shaped as "make X scalable for Y" with X = path-sensitive pointer analysis and Y = SVFA — the standard precision/scalability frontier paper of the year.

Objective Summary

- **Core idea novelty:** Re-engineers path-sensitive pointer analysis so that the path constraints it produces are *consumable by* sparse value-flow clients — not just by alias clients. The pivotal idea is to push path conditions into value-flow edges rather than into points-to facts, narrowing the SVFA query frontier with path information.
- **Methodology novelty:** Marries Shi-lab's prior compositional, on-demand pointer analysis with a sparse-value-flow query schema; the compositional summaries become the unit of path-sensitive composition rather than statement-level guard formulas.
- **Evaluation findings and insights (anticipated from venue norms):** Wins on real-world C/C++ benchmarks with order-of-magnitude speedups vs path-sensitive baselines (PSA + SVFA) at preserved precision; the bottleneck shifts from constraint solving to query scheduling.

Meta-cognition Analysis

- **Problem formulation:** "Path sensitivity is needed for precision in vuln detection, but adding it to SVFA blows up SAT calls." The authors choose to formalize the cost in terms of *where* path conditions are emitted and *which client* consumes them.
- **Solution-space reduction:** Rather than improve SAT/SMT speeds (saturated direction) or coarsen path sensitivity (loses precision), the design narrows to *redistributing* path conditions along sparse value-flow edges, so a client only solves the conditions it needs.
- **Key assumptions:** SVFA queries are sparse and locally bounded; demand-driven path conditions can be cached by summary keys.
- **Main trick / routine:** Compositional summaries keyed by entry/exit value-flow facts + per-summary path predicate; client materializes only relevant predicates at query time.
- **How the method was likely derived:** Diff between "we have path-sensitive pointer analysis (Pinpoint) and we have SVFA (SVF)" frame → ask why their composition isn't path-sensitive in practice → identify that SVFA's frontier discards path info → fix by attaching path predicates to value-flow edges.
- **Reusable research pattern:** *Annotate-and-defer*: instead of eagerly resolving expensive precision dimensions, attach them as edge labels and let the client resolve only labels it crosses.
- **Potential inspiration:** Apply *annotate-and-defer* to taint analysis, type-state analysis, or LLM-assisted vuln retrieval (path predicates as retrieval keys).

Critical Assessment

- **Strengths:** Real engineering on the precision/scale frontier; PACMPL-grade rigour; Shi-group benchmarks are reproducible.
 - **Open questions:** Cross-language transfer (beyond C/C++) and soundness interaction with LLVM IR optimization passes are natural follow-up directions.
 - **What is genuinely valuable:** The *recipe* — how to compose two existing precision dimensions without paying for both at every program point.
-

2.2 Prunario: Testing Autonomous Driving Systems by Pruning Likely Redundant Scenarios

Paper

- **Title:** Prunario: Testing Autonomous Driving Systems by Pruning Likely Redundant Scenarios
- **Authors:** M. Kim, S. So, H. Oh
- **Venue / Source:** PACMPL 2026 (PLDI/OOPSLA-class)
- **Year:** 2026

Priority Assessment

- **Source Priority:** High — Hakjoo Oh (followed) + PACMPL.
- **Reason:** Both author and venue qualify; topic (search-based testing with redundancy pruning) is directly reusable for fuzzing campaigns.

Screening Decision

- **Stage 1 Decision:** Proceed
- **Reason:** “Pruning likely-redundant scenarios” is the core unsolved problem in scenario-based AV testing — it directly attacks the test-budget-vs-coverage bottleneck.
- **Research Value:** High
- **Methodology Reusability:** High (the redundancy classifier is portable to any expensive simulator).

Quality Decision

- **Decision:** Outstanding
- **Reason:** Tackles the exact pain point of AV testing budgets, in a venue that demands rigour about the redundancy oracle’s soundness/completeness.

Objective Summary

- **Core idea novelty:** A “likely-redundant” predicate over scenarios that is computed *before* simulation, pruning the search frontier. The novelty is choosing a predicate cheap enough to be useful but tight enough to preserve discovered failures.
- **Methodology novelty:** Likely a learned similarity over scenario specifications combined with diversity-aware pruning, with a calibrated false-prune rate.
- **Evaluation findings and insights:** Expected: same number of unique failures discovered with substantially fewer simulator runs; the redundancy threshold’s elbow becomes a tunable knob.

Meta-cognition Analysis

- **Problem formulation:** “Most generated AV scenarios in test campaigns trigger the same failure pattern; we burn simulator hours on redundancy.”
- **Solution-space reduction:** Instead of post-hoc deduplication on traces (expensive) or coarse spec-distance (imprecise), use a learned predicate on *specification structure* + a counterexample-driven calibrator.
- **Key assumptions:** Failure-equivalence has an upstream signal in scenario parameters; redundancy distribution is approximately stationary across campaigns.
- **Main trick / routine:** Surrogate redundancy classifier + budget-aware stop rule.
- **How the method was likely derived:** Observed huge mass of duplicate failures in baseline (e.g., DriveFuzz / SAMOTA) → ask “could we predict redundancy from spec alone” → train classifier on small simulated batch → integrate as pre-filter.
- **Reusable research pattern:** *Surrogate-then-prune* — train a cheap predictor of expensive oracle output and prune before paying.
- **Potential inspiration:** Same pattern fits LLM-driven fuzzing of compilers, kernel fuzzing seed selection, and program-analysis warning triage.

Critical Assessment

- **Strengths:** Matches a real industrial pain point; PACMPL acceptance signals the formal redundancy guarantees are stated carefully.
- **Open questions:** Sensitivity to scenario-domain coverage and the false-prune rate curve under distribution shift are natural follow-ups.
- **What is genuinely valuable:** The framing (pruning *likely* redundant), the calibration argument, the integration into search loops.

2.3 Too Private to Tell: Practical Token Theft Attacks on Apple Intelligence (Serpent)

Paper

- **Title:** Too Private to Tell: Practical Token Theft Attacks on Apple Intelligence
- **Authors:** Haoling Zhou, Shixuan Zhao, Chao Wang, Zhiqiang Lin (Ohio State)
- **Venue / Source:** arXiv:2604.15637 (cs.CR), submitted 2026-04-17

- **Year:** 2026
- **Link:** [arXiv:2604.15637](https://arxiv.org/abs/2604.15637)

Priority Assessment

- **Source Priority:** High — Zhiqiang Lin (followed). Apple has confirmed and assigned a CVE (vendor-validated).
- **Reason:** Top followed researcher + concrete vendor-acknowledged vulnerability + first-of-its-kind class of attack on a mainstream AI service.

Screening Decision

- **Stage 1 Decision:** Proceed
- **Reason:** Real-world impact, CVE issued, novel attack class on a closed system reverse-engineered through traffic analysis.
- **Research Value:** High
- **Methodology Reusability:** High — methodology (traffic analysis + RE + cross-comparison with public docs) generalises to *any* closed AI service with anonymous access tokens.

Quality Decision

- **Decision:** Outstanding
- **Reason:** Validated by vendor, novel attack primitive, clean lesson for system designers.

Objective Summary

- **Core idea novelty:** First practical *cross-device token replay* against Apple Intelligence — the “Serpent” attack steals an anonymous access token on one device and reuses it on another, *with the rate-limit charged to the victim*. Demonstrated on macOS 26 Tahoe.
- **Methodology novelty:** A black-box reverse-engineering pipeline: traffic capture → reverse engineering of the on-device authn/authz handshake → comparison with Apple’s public security documentation to localize the gap → exploit construction. The attack lives in the gap between “what the documentation claims” and “what the protocol enforces.”
- **Evaluation findings and insights:** Even after the attacker exhausts their own quota, replayed tokens immediately restore access. CVE assigned, bounty awarded. The systemic lesson: *anonymising* identity is not the same as *binding* it — non-transferability requires cryptographic binding to the rightful user.

Meta-cognition Analysis

- **Problem formulation:** “Apple claims privacy via two-stage anonymous tokens. Anonymity ≠ unforgeability. Are tokens bound to anything device-specific?”
- **Solution-space reduction:** Don’t try to break the cryptography of the AI service; instead probe the *binding* of the access token to the issuing device. This collapses a protocol-analysis problem into a few targeted experiments.

- **Key assumptions:** Apple's public documentation is roughly accurate at the high level — useful as a hypothesis source; macOS network traffic is observable enough through standard MITM tooling.
- **Main trick / routine:** Document-vs-protocol diff. Read the formal claim, model what enforcement *would* look like, test whether the wire actually enforces it.
- **How the method was likely derived:** Authors likely started from the published Apple Privacy paper / security guide claiming anonymous tokens, asked “what would break anonymity vs non-transferability?”, noticed transferability is rarely cryptographically enforced in token-issuance flows, then built the cross-device experiment.
- **Reusable research pattern:** *Claim-vs-Wire diff*. Treat vendor security claims as a hypothesis space; the contribution is showing where the wire diverges. Repeatable for: confidential computing services, anonymous credential systems, Private Cloud Compute, on-device-LLM frameworks.
- **Potential inspiration:** Apply *claim-vs-wire diff* to local LLM frameworks claiming “data never leaves device” and to enterprise GenAI gateways that promise per-tenant rate-limits.

Critical Assessment

- **Strengths:** Vendor-confirmed CVE; clean separation between conceptual lesson (anonymisation $\neq$ non-transferability) and technical attack; directly actionable mitigations.
- **Open questions:** How broadly the lesson generalises across other built-in AI services would benefit from a follow-up survey.
- **What is genuinely valuable:** The *taxonomy distinction* between anonymisation and non-transferability — this is reusable as a checklist item in future security reviews.

Clarity of Rationale (outstanding paper)

- The abstract structures itself as: *what the system claims* → *what we found* → *what general lesson follows*. That single-paragraph contour is a transferable writing pattern.

Exemplary Writing Traits

- Names the attack (*Serpent*) and the bug class (*cross-device token replay*) — both stick.
- Closes with a one-sentence general lesson (“Anonymising identity does not by itself make the AI service secure; enforcing non-transferability requires cryptographic binding to the rightful user.”). This is a textbook *kicker*.

2.4 Evaluating LLM Agents on Automated Software Analysis Tasks (AnalysisBench)

Paper

- **Title:** Evaluating LLM Agents on Automated Software Analysis Tasks
- **Authors:** Islem Bouzenia, Cristian Cadar, Michael Pradel
- **Venue / Source:** arXiv:2604.11270 (cs.SE), v2 2026-04-17

- **Year:** 2026
- **Link:** [arXiv:2604.11270](https://arxiv.org/abs/2604.11270)

Priority Assessment

- **Source Priority:** High — Pradel (followed) + Cadar; Pradel-Cadar collaborations consistently land at top SE venues.
- **Reason:** Strong author signal; benchmark+agent papers in this lineage tend to become reference points.

Screening Decision

- **Stage 1 Decision:** Proceed
- **Reason:** Frames a *new* benchmark task class (automated software analysis $\neq$ issue solving $\neq$ environment setup) and produces head-to-head numbers with manual ground truth.
- **Research Value:** High
- **Methodology Reusability:** High — the benchmark + the architectural lessons (stage mixing, error localization, premature termination) are directly applicable to any agentic SE evaluation.

Quality Decision

- **Decision:** Outstanding
- **Reason:** Clean problem definition + 35 hand-crafted ground-truth tool-project pairs + four agent architectures $\times$ four LLM backends + the *meta* finding that “agentic architecture matters more than LLM capability alone” is a reusable claim.

Objective Summary

- **Core idea novelty:** Carves out *automated software analysis* as a distinct agentic task: install + configure an analysis tool *alongside* a target project, generate tool-specific prerequisites, and validate that the tool produces meaningful output. Unlike SWE-bench (issue-solving) or repo-environment-setup, success requires a *triadic* setup (target + tool + glue).
- **Methodology novelty:** AnalysisBench = 35 tool-project pairs over 7 analysis tools and 10 C/C++/Java projects, each with manually constructed reference setups. AnalysisAgent achieves 94% manual-verified success (Gemini-3-Flash) vs 77% for ExecutionAgent baseline.
- **Evaluation findings and insights:**
 - LLM-self-validated success consistently *overstates* manually verified success (recurring honesty problem of agents grading themselves).
 - Whole-program analyses and symbolic execution are the hardest tasks.
 - Java toolchains pose greater challenges than C/C++ (build-system surface area).
 - Agentic *architecture* dominates LLM *capability*: a smarter agent on a weaker model often beats a dumber agent on a stronger model.

Meta-cognition Analysis

- **Problem formulation:** “Agentic SE benchmarks conflate issue-solving and analysis-tool-running. Build a benchmark that isolates the second.”
- **Solution-space reduction:** Pick a manageable but representative basket — 7 tools × 10 projects with hand-curated reference setups — rather than auto-mining. Manual ground truth is the key reduction step that lets you separate *self-validated* from *real* success.
- **Key assumptions:** Reference setups are achievable for a known maximum complexity; manual verification scales to ~140 (4×35) trial outcomes.
- **Main trick / routine:** *Triadic ground truth*. Don’t measure agent success on a binary — measure (a) tool installs, (b) target builds, (c) tool produces meaningful output. Each is a separate signal.
- **How the method was likely derived:** Failure analysis on existing agent benchmarks → notice “tool-running” is a sub-task that dominates real-world utility but has no benchmark → carve it out. The four limitations identified (stage mixing, poor error localization, premature termination, self-validation) almost certainly came from manual error inspection.
- **Reusable research pattern:** *Decompose agent success*. When benchmarking an agent, decompose its task into independent sub-signals; binary success masks the bottlenecks.
- **Potential inspiration:** Same decomposition trick for LLM-driven fuzzing harness synthesis: separate “harness compiles” / “harness runs” / “harness reaches target” / “harness finds bug”.

Critical Assessment

- **Strengths:** Clean benchmark scope; manual verification gives high-trust numbers; the “architecture > model” finding is empirically falsifiable and policy-shaping.
- **Open questions:** How the architecture-vs-capability finding evolves with stronger frontier models; how the benchmark generalizes to OSS LLM backends.
- **What is genuinely valuable:** The four agent failure modes (stage mixing, poor error localization, premature termination, self-validation overstatement) — these become a checklist for any future agentic-SE design.

Clarity of Rationale (outstanding paper)

- Frames itself as “*unlike issue solving or general environment setup, [analysis] requires X, Y, Z*” — explicitly differentiating from neighbouring benchmarks. This *contrast-define* opening is highly portable.

Exemplary Writing Traits

- Names the artifacts (*AnalysisBench*, *AnalysisAgent*) and the failure modes; nameable concepts compound citations.
 - Reports both *self-validated* and *manually verified* success. This dual reporting will likely become a community norm on the strength of this paper alone.
-

3. Keep Papers (Stage-2 brief deep-read)

3.1 TitanCA: Lessons from Orchestrating LLM Agents to Discover 100+ CVEs

- **Authors:** T. Zhang, Y. Li, C. Yang, R. Widyasari, Y. Liu, NT Bui, ... (David Lo group)
- **Venue:** arXiv 2026
- **Source priority:** High (David Lo followed).
- **Decision:** Keep.

Core idea: Orchestration framework where a colony of specialised agents (call-graph triage, taint sketch, exploit synth, validation) is run over real codebases, discovering 100+ confirmed CVEs.

Reusable pattern: *Pipeline beats monolith* — divide the agentic vuln-finding loop into stages whose individual error rates can be measured, then optimise the bottleneck stage.

Genuine value: A field study at this scale supplies priors on which agent stages actually pay off, which is exactly the missing data in the Pradel paper above.

3.2 OS-SANITIZER: System-wide Latent Defect Inference in Linux Applications

- **Authors:** A. Crump, S. Sihag, F. Bauckholt, K. Hassler, T. Holz
- **Venue:** preprint 2026 (Holz group → likely USENIX Sec / NDSS submission target)
- **Source priority:** High (Thorsten Holz followed; CISPAs is a top group).
- **Decision:** Keep.

Core idea: Extend dynamic-testing defect detection from “did it crash?” to “is there a latent defect that did not crash this run?” — system-wide across Linux applications.

Reusable pattern: *Latent-state oracles* — replace coarse “crashed” oracles with finer invariants over kernel/userland state to surface bugs that survived the run.

3.3 Determining the Unreachable: Constraint-Guided Reachability Analysis for Dependency Vulnerabilities

- **Authors:** W. Feng, X. Li, R. Feng, Y. Zhang, Y. Li, Z. Zhou, Y. Wang, ...
- **Venue:** PACMPL 2026
- **Source priority:** Borderline High-Priority (Recommended alert, but PACMPL = top venue → upgrade).
- **Decision:** Keep (Borderline High-Priority).

Core idea: Tackle the SCA “false-positive-by-default” problem — most reported CVE matches in dependencies are *unreachable* from the application. Use constraint-guided reachability to filter them out.

Reusable pattern: *Negative reachability as triage*. Most static analysis tools are designed to flag positives; here the value is in *certifying* unreachability of vulnerable functions, which is the true industrial workflow.

3.4 V2E: Validating Smart Contract Vulnerabilities through Profit-driven Exploit Generation and Execution

- **Authors:** J. Zhang, Y. Nan, K. Ning, M. Ye, W. Li, Y. Xiao, Y. Feng, ...
- **Venue:** arXiv 2026 (Zibin Zheng group — Sun Yat-sen)
- **Source priority:** High (Zibin Zheng followed).
- **Decision:** Keep.

Core idea: Move the smart-contract bug oracle from “could this be exploited?” to “was it actually profitably exploited on-chain?” — i.e., generate executable PoCs whose profit > 0 .

Reusable pattern: *Profit-as-oracle*. In economic systems, the cleanest correctness oracle is “does the attacker actually gain?” rather than “is the post-state inconsistent?”.

Genuine value: Decouples vulnerability *existence* from *exploitability*, sharply reducing the warning flood that has plagued smart-contract analysis.

3.5 Fragile Deliveries: Inconsistencies in Android Parcel and Their Security Consequences

- **Authors:** H. Chen, C. Wang, Y. Yang, J. Miller, T. Bao, R. Wang, ...
- **Venue:** preprint 2026 (Zhiqiang Lin co-author lineage; ASU/OSU)
- **Source priority:** High (Zhiqiang Lin followed).
- **Decision:** Keep.

Core idea: Audit Parcel-based IPC on Android for serializer/deserializer asymmetries — i.e., the writing side and reading side diverge in interpretation. Such inconsistencies are a classic source of confused-deputy and type-confusion bugs.

Reusable pattern: *Differential serializer audit*. Parse-don't-validate failures generalise; same audit applies to JSON/Protobuf in cloud RPC, gRPC server/client mismatches, kernel ioctl marshallers.

3.6 Denoising Fault Localization with Test Line Proximity

- **Authors:** M. Smytzek, A. Zeller
- **Venue:** preprint 2026 (CISPA / Zeller group)
- **Source priority:** High (Andreas Zeller followed).
- **Decision:** Keep.

Core idea: Statistical Fault Localisation (SFL) is famously noisy — many spurious top-ranked statements. Use *test-line proximity* (graph distance between failing tests and source lines in some structural sense) as a denoiser on top of standard SBFL formulae.

Reusable pattern: *Structural prior over statistical scores*. When a statistical estimator is noisy and you have a cheap structural prior, multiply (rather than replace).

4. Borderline High-Priority Papers

4.1 FAUDITOR — Capturing Monetarily Exploitable Vulnerability in Smart Contracts via Auditor Knowledge-Learning Fuzzing

- **Authors:** Bowen Cai, Weiheng Bai, Hangyun Tang, Youshui Lu, Kangjie Lu
- **Venue:** arXiv:2604.18395 (cs.CR), 2026-04-20
- **Link:** [arXiv:2604.18395](https://arxiv.org/abs/2604.18395)
- **Source priority:** Borderline High-Priority (Recommended alert, but Kangjie Lu / Minnesota is a high-credibility security group).

Objective summary: Formalises *Monetarily Exploitable Vulnerabilities* (MEVuls) in DeFi (price manipulation, inflation attacks). FAUDITOR is a fuzzer that (a) targets finance-related interfaces directly, (b) uses NLP over auditor reports to extract exploitation patterns, and (c) self-refines its strategies from prior fuzzing results. Reports 220 zero-day MEVuls discovered.

Meta-cognition: - **Problem formulation:** Existing fuzzers either drown developers in warnings or under-cover financial logic. Restrict the fuzz domain to finance interfaces. - **Solution-space reduction:** Use *auditor reports* as a corpus for exploitation patterns — the labels already exist in audit literature. - **Reusable pattern:** *Practitioner-knowledge mining*. Where a community of human experts has already produced systematic write-ups (audits, advisories, postmortems), mine them as labeled supervision for a search-based tool.

4.2 Cracking Federated Privacy: Initialization-Resilient Gradient Inversion with Fine-Grained Reconstruction

- **Authors:** K. Zhu, J. Yang, S. Guo, H. Qin, T. Wang, J. Wang, Y. Nan, ...
- **Venue:** preprint 2026
- **Source priority:** Borderline High-Priority.

Core idea: Strengthen gradient-inversion attacks against FL by removing the brittle “favourable initialization” assumption that has haunted prior attacks.

Reusable pattern: *Assumption removal as contribution*. Prior attacks worked under unrealistic init; the contribution is *operating without it*. This is a classic axis for follow-up papers in adversarial ML.

5. Cross-Paper Synthesis — Themes & Patterns

5.1 Three converging threads this window

1. **Agentic SE is moving from existence proofs to architecture science.** Pradel's *AnalysisBench* and David Lo's *TitanCA* together signal the maturation: the question is no longer “can an LLM agent do X?” but “what architecture, error-localization, and self-validation policy are needed for X?” The *self-validation overstatement* finding from Pradel and the *pipeline orchestration* lessons from *TitanCA* are the two pieces that future agentic papers will be measured against.

2. **Static analysis is doubling down on precision–scalability composition.** Hermes (Shi) on path-sensitive SVFA and “Determining the Unreachable” (PACMPL) on dependency reachability are both *composition* papers — the contribution isn’t a new analysis, it’s making two existing dimensions cohabit at scale. Recipe: attach precision artefacts as edge labels (Hermes) or reachability constraints (Unreachable), defer expensive resolution to query time.
3. **Security research is rewarding `claim-vs-wire diff` methodology.** Lin’s *Serpent* attack on Apple Intelligence and *Fragile Deliveries* on Android Parcel are the same methodological move applied to different layers: read the vendor’s documented contract, model what enforcement would look like, find where the wire diverges. This pattern is highly portable to PCC, on-device LLMs, and confidential-computing services.

5.2 Reusable research patterns extracted this week

Pattern	Donor paper(s)	Applicable to
Annotate-and-defer (push precision into edge labels, resolve only on traversal)	Hermes	Taint analysis, type-state, retrieval-keyed LLM analysis.
Surrogate-then-prune (cheap classifier of expensive oracle output)	Prunario	Compiler fuzzing, kernel fuzzing seed selection, warning triage.
Claim-vs-wire diff (treat vendor security claims as hypothesis space)	Serpent, Fragile Deliveries	PCC, on-device LLMs, enterprise GenAI gateways.
Decompose agent success (separate sub-signals: install / build / run / output)	AnalysisBench	Harness-synthesis fuzzing, repo migration agents, exploit-generation agents.
Profit-as-oracle (use economic gain as the correctness oracle in financial systems)	V2E	DeFi audit, MEV bot defense, exchange-protocol testing.
Negative reachability as triage (certify unreachability rather than enumerate positives)	Determining the Unreachable	SCA pipelines, alert deduplication.
Practitioner-knowledge mining (mine human-curated audits/advisories as labels)	FAUDITOR	Fuzzing seeds, warning prioritisation, vuln-class prediction.
Structural prior × statistical estimator	Denosing FL	LLM ranking denoising, FP triage.

Pattern	Donor paper(s)	Applicable to
Differential serializer audit	Fragile Deliveries	RPC, kernel ioctl, IPC frameworks.

5.3 Trend signals

- **PACMPL is absorbing what used to be ICSE/FSE program-analysis papers.** Three of our top six are PACMPL.
- **CVE-count headlines are becoming a load-bearing claim.** Both TitanCA (100+ CVEs) and FAUDITOR (220 zero-days). We should normalise *unique-class* CVE counts in our own future papers.
- **Vendor confirmation as a quality signal.** Serpent’s CVE-and-bounty closes the credibility gap that pure-academic security papers historically fight; expect more of this in 2026.

6. Writing & Rationale Insights (for our own papers)

Drawing on the four Outstanding papers:

1. **Name your artifacts and your bug class.** *Hermes, Prunario, Serpent, AnalysisBench/AnalysisAgent, MEVuls.* Nameable concepts compound citations and make follow-up work easier to position.
2. **Open with a contrast-define paragraph.** Pradel et al. don’t just describe analysis-tool-running; they *contrast it against* SWE-bench-style issue solving and against generic env-setup. Readers retain a definition pinned by what it isn’t.
3. **Report dual metrics when self-evaluation is in play.** AnalysisBench reports both LLM-self-validated and manually-verified success. This is an honesty signal; the gap itself is a contribution.
4. **Close with a one-sentence general lesson.** Serpent’s “Anonymising identity does not by itself make the AI service secure; enforcing non-transferability requires cryptographic binding to the rightful user.” That kicker travels further than the technical bug.
5. **Solution-space reduction is the contribution.** Hermes doesn’t make SAT faster; it makes SAT *less needed*. Prunario doesn’t accelerate the simulator; it *avoids* it. The discipline is: for each candidate solution direction, articulate the saturation argument *first*, then justify the unsaturated direction.

End of public report — 2026-04-26.